

Retrospektywa

Jak projektować i mierzyć pracę programisty z AI

Retrospektywa.pl

Spis treści

Retrospektywa	7
Książka jako laboratorium	7
Pobierz wydanie 0.2	8
Jak czytać	8

I

I. Człowiek i przepływ pracy

1	Programista jako wąskie gardło	11
1.1	Uwaga jako budżet	11
1.2	Sygnały przeciążenia	11
1.3	Eksperyment	11
2	Stanowisko pracy	13
2.1	Dwa monitory, dwie funkcje	13
2.2	Przeglądarka jako narzędzie źródłowe	13
2.3	Dwie aplikacje AI	13
2.4	Higiena stanowiska	14
2.5	Eksperyment	14
3	Kolejka zamiast chaosu	15
3.1	Minimalna karta zadania	15
3.2	Stany kolejki	15
3.3	Zasady odbioru	15
3.4	Backpressure	15
3.5	Eksperyment	16
4	Dwa modele, różne role	17
4.1	Podział odpowiedzialności	17
4.2	Kiedy drugi model pomaga	17
4.3	Kiedy drugi model szkodzi	17
4.4	Routing po capability	17
4.5	Eksperyment	18
5	Równoległość bez multitaskingu	19
5.1	Pipeline zamiast wielu otwartych frontów	19
5.2	Dopuszczalny drugi projekt	19
5.3	Limity pracy w toku	19
5.4	Eksperyment	20

II	II. Warsztat i pomiar	
6	Shell jako centrum operacyjne	23
6.1	Izolacja zadania	23
6.2	Kontrakt poleceń	23
6.3	Sesja i receipt	23
6.4	Bezpieczeństwo	24
6.5	Eksperyment	24
7	Jak mierzyć efektywność	25
7.1	Minimalny zestaw	25
7.2	Jednostka porównania	25
7.3	Metryki ostrzegawcze	25
7.4	Czego nie nagradzać	26
7.5	Karta wyniku	26
8	Budżet i ograniczenia AI	27
8.1	Cztery budżety	27
8.2	Budżet zadania	27
8.3	Dwa dostępy jako odporność	27
8.4	Plan degradacji	27
8.5	Eksperyment	28
III	III. System uczący się	
9	Digital Twin Portrait, router i orchestrator ticketów	31
9.1	Intencja skrócona do kontraktu	31
9.2	Łańcuch wykonania	31
9.3	Dowód z próby wdrożeniowej	32
9.4	Eksperyment dla własnego workflow	32
9.5	Wniosek	32
10	DSL: idea, którą można uruchomić	33
10.1	Języki i odpowiedzialności	33
10.1.1	Kwalifikuj profile	33
10.2	Kompresja bez utraty znaczenia	34
10.3	Minimalny przebieg	34
10.4	Process Pack	34
10.5	Reguła ewolucji	35
10.6	Granica między przykładem a runtime	35
11	Cotygodniowa retrospektywa	37
11.1	Dane wejściowe	37
11.2	Szablon	37

11.3	Cztery pytania	37
11.4	Pierwsze trzydzieści dni	38

IV IV. Architektura wykonywalna

12	Od SOA do POA: URI Process	41
12.1	URI jest nazwą, nie transportem	41
12.2	Kontrakt procesu	42
12.3	Od bindings do dowodu	42
12.4	Przykład publikacji	42
12.5	Granice bezpieczeństwa	43
12.6	Eksperyment	43
13	Książka jako wykonywalny system publikacji	45
13.1	Jedno źródło, różne projekcje	45
13.2	Audio jako artefakt	45
13.3	Reprodukowalność	46
13.4	Test wydania	46
14	Forma współpracy, authority i dowód pracy	47
14.1	Główne formy współpracy	47
14.2	Lifecycle relacji	48
14.3	Ile może organizować pracodawca	48
14.4	Proces zamiast totalnego monitoringu	49
14.5	Granica między UoP a relacją cywilną	50
14.6	Wniosek dla modelu actor/agent	50

V Aneksy

15	Mapa badań i siła dowodu	55
15.1	Jak interpretować wyniki	55
16	Process Pack do pobrania	57
16.1	Jak pracować z przykładem	57
	Bibliografia	59

Retrospektywa

Jak projektować i mierzyć pracę programisty z AI

Efektywność programisty pracującego z AI zależy nie od szybkości generowania kodu, lecz od sposobu zarządzania uwagą, kolejką zadań, weryfikacją wyników i ograniczonym budżetem modeli.

Modele potrafią przygotować więcej propozycji, niż człowiek jest w stanie odpowiedzialnie ocenić. W takim środowisku kolejne okno czatu nie zwiększa automatycznie produktywności. Może jedynie powiększyć kolejkę nieodebranej pracy.

Ta książka pokazuje, jak zbudować system, w którym:

- człowiek zachowuje jedno aktywne zadanie wymagające skupienia,
- modele wykonują rozdzielone i możliwe do zweryfikowania role,
- wyniki trafiają do kolejki zamiast przerywać pracę,
- koszt, czas, poprawki i jakość są mierzone,
- autonomia rośnie dopiero po zebraniu powtarzalnych dowodów.

Nie jest to katalog promptów ani ranking dostawców. Nazwy modeli, limity i ceny zmieniają się szybciej niż dobra organizacja pracy. Interesują nas mechanizmy, które pozostają użyteczne po zmianie narzędzia: kontrakt celu, ograniczenie pracy w toku, niezależna weryfikacja, jawny budżet i retrospektywa.

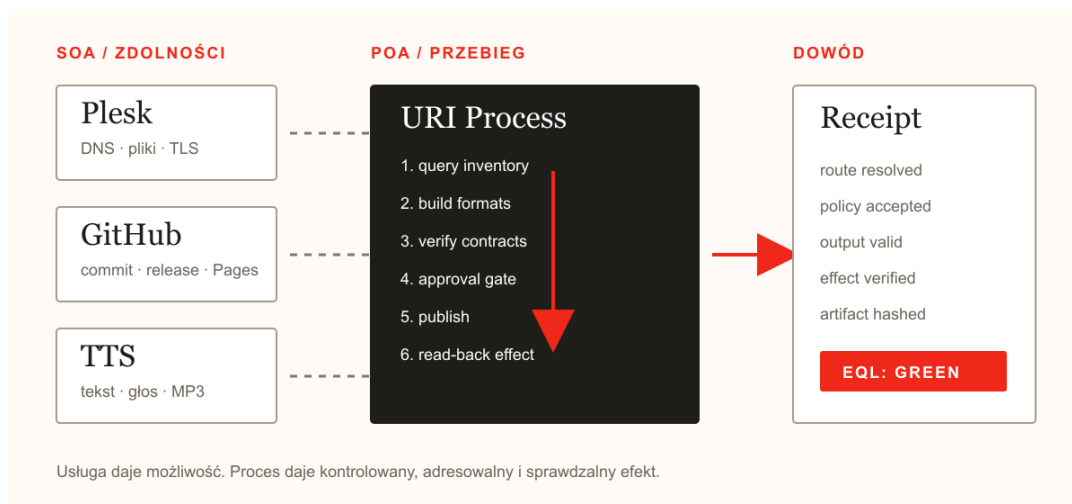
Książka jako laboratorium

Każdy rozdział łączy narrację z małym, odtwarzalnym eksperymentem. Część decyzji zapisujemy również za pomocą DSL-i wywodzących się z projektów Subactora:

Warstwa	Pytanie
Intent Contract	Co dokładnie chcemy osiągnąć?
AQL	Kto może wykonać działanie i czego nie wolno?
OQL	Jaki jest przebieg operacji?
URI Process	Który adresowalny proces realizuje krok i przez jaki runtime?
EQL i TestQL	Po czym poznamy prawidłowy rezultat?
DOQL i DQL	Jaka jest sytuacja i które inwarianty zostały naruszone?
SODL	Co faktycznie wydarzyło się w czasie?

DSL nie zastępuje prozy. Proza wyjaśnia motywację i kontekst, a kontrakt oddziela pola, które można walidować, porównywać i testować.

Wydanie 0.2 rozwija ten model o przejście od architektury usługowej do architektury zorientowanej na proces. Książka, audiobook, przykłady i strona nie są osobnymi produktami redakcyjnymi: powstają z jednego wersjonowanego źródła oraz wspólnego generatora.



Rysunek 0.1: SOA udostępnia zdolności, POA składa je w kontrolowany proces, a URI Process wiąże plan z runtime'em i dowodem.

Pobierz wydanie 0.2

- [PDF](#)
- [EPUB](#)
- [DOCX](#)
- [próbka audiobooka MP3](#)
- [Process Pack: DSL, schematy i przykłady](#)
- [manifest wydania z sumami SHA-256](#)

Jak czytać

1. Wybierz jeden rzeczywisty proces, nie całą pracę naraz.
2. Zapisz stan początkowy przez co najmniej kilka porównywalnych zadań.
3. Wprowadź jedną zmianę i z góry określ czas trwania próby.
4. Porównaj jakość, koszt, czas oraz aktywny udział człowieka.
5. Zachowaj zmianę tylko wtedy, gdy wynik poprawił cały system, a nie jedną wygodną metrykę.

Po każdym eksperymencie powinien pozostać ślad: kontrakt, zdarzenia, wynik i decyzja. Dzięki temu retrospektywa przestaje być opinią z końca tygodnia i staje się metodą rozwijania sposobu pracy.

I

I. Człowiek i przepływ pracy

1	Programista jako wąskie gardło	11
1.1	Uwaga jako budżet	11
1.2	Sygnaly przeciążenia	11
1.3	Eksperyment	11
2	Stanowisko pracy	13
2.1	Dwa monitory, dwie funkcje	13
2.2	Przeglądarka jako narzędzie źródłowe	13
2.3	Dwie aplikacje AI	13
2.4	Higiena stanowiska	14
2.5	Eksperyment	14
3	Kolejka zamiast chaosu	15
3.1	Minimalna karta zadania	15
3.2	Stany kolejki	15
3.3	Zasady odbioru	15
3.4	Backpressure	15
3.5	Eksperyment	16
4	Dwa modele, różne role	17
4.1	Podział odpowiedzialności	17
4.2	Kiedy drugi model pomaga	17
4.3	Kiedy drugi model szkodzi	17
4.4	Routing po capability	17
4.5	Eksperyment	18
5	Równoległość bez multitasking	19
5.1	Pipeline zamiast wielu otwartych frontów	19
5.2	Dopuszczalny drugi projekt	19
5.3	Limity pracy w toku	19
5.4	Eksperyment	20

1. Programista jako wąskie gardło

AI może generować więcej wyników, niż człowiek jest w stanie odpowiedzialnie odebrać. Dlatego podstawową jednostką systemu nie jest prompt ani liczba wygenerowanych linii, lecz **decyzja zakończona dowodem**.

Badania nad wznawianiem przerwanych zadań programistycznych pokazują, że powrót do kodowania wymaga rekonstrukcji planu, intencji i kontekstu pracy [1]. W badaniu postrzegania produktywności programiści łączyli produktywny dzień z domykaniem znaczących zadań bez nadmiernych przerw i zmian kontekstu [2].

W typowym zadaniu człowiek nadal musi:

1. rozpoznać problem i jego granice,
2. określić kryteria akceptacji,
3. udostępnić właściwy kontekst,
4. ocenić proponowane rozwiązanie,
5. rozstrzygnąć sprzeczności,
6. zaakceptować ryzyko i rezultat.

Model może skrócić część wykonawczą, ale równocześnie tworzy nową pracę: propozycje trzeba przeczytać, porównać, uruchomić i często poprawić. Jeśli generowanie przyspiesza bardziej niż odbiór, kolejka rośnie. System wygląda na zajęty, lecz czas od pomysłu do zaakceptowanej zmiany się nie poprawia.

1.1 Uwaga jako budżet

Uwaga nie jest jedną liczbą minut. Ma co najmniej trzy koszty:

- **wejście w kontekst** — przypomnienie celu, ograniczeń i stanu kodu,
- **utrzymanie kontekstu** — praca bez nieplanowanych przerw,
- **domknięcie decyzji** — sprawdzenie dowodu i zapisanie następnego kroku.

Najdroższa jest praca, która została rozpoczęta, ale nie zakończyła żadnej decyzji. Dlatego przyjmujemy prosty limit:

Human WIP = 1: człowiek ma jedno aktywne zadanie poznawcze. Pozostałe zadania mogą czekać albo być wykonywane przez automaty, lecz nie konkurują o uwagę.

Limit nie oznacza, że system realizuje tylko jedną rzecz. Oznacza, że człowiek nie próbuje jednocześnie utrzymywać kilku pełnych kontekstów.

1.2 Sygnały przeciążenia

System jest przeciążony, gdy:

- nowe odpowiedzi modeli regularnie przerywają aktywną pracę,
- kilka diffów czeka na review dłużej niż trwała ich implementacja,
- człowiek ponownie czyta ten sam kontekst,
- zadania są „prawie gotowe”, ale nie mają testu ani decyzji,
- liczba rozpoczętych zmian rośnie szybciej niż liczba zaakceptowanych.

1.3 Eksperyment

Przez pięć dni utrzymuj limit jednej aktywnej decyzji człowieka. Każdy wynik AI trafia do kolejki odbioru i nie wywołuje powiadomienia.

Zapisuj:

- czas rozpoczęcia i zakończenia aktywnej pracy,
- liczbę nieplanowanych przełączeń,
- czas oczekiwania wyniku na review,
- liczbę powrotów do wcześniej otwartego zadania,
- liczbę zaakceptowanych zmian.

Nie próbuj jeszcze optymalizować modeli. Najpierw sprawdź, czy sama ochrona uwagi skraca czas domykania decyzji. Sukcesem nie jest większa liczba odpowiedzi AI, lecz krótsza kolejka przy co najmniej tej samej jakości.

2. Stanowisko pracy

Pierwszy monitor służy aktywnemu problemowi. Drugi pokazuje dokumentację, kolejkę wyników AI i stan automatycznych kontroli. Przeglądarka jest narzędziem źródłowym, a terminal — centrum operacyjnym.

Sprzęt nie tworzy metody, ale może ją wspierać albo sabotować. Celem stanowiska nie jest pokazanie jak największej liczby aplikacji. Celem jest utrzymanie czytelnej granicy między **pracą aktywną** a **pracą oczekującą**.

2.1 Dwa monitory, dwie funkcje

Monitor aktywny zawiera tylko to, co jest potrzebne do bieżącej decyzji:

- edytor albo diff,
- terminal z testem dotyczącym tej zmiany,
- jedno okno rozmowy związane z zadaniem.

Monitor pomocniczy pokazuje informacje, które można bezpiecznie odłożyć:

- dokumentację i źródła,
- kolejkę zadań oraz wyników,
- statusy buildów i testów,
- licznik budżetu,
- notatkę przekazania kontekstu.

Dwa monitory nie oznaczają dwóch zadań poznawczych. Oznaczają mniej niepotrzebnych przełączeń podczas jednego zadania.

2.2 Przeglądarka jako narzędzie źródłowe

Przeglądarka jest nieodłączna od pracy, ale nie powinna być przypadkową stertą kart. Dla aktywnego zadania warto utrzymywać trzy grupy:

1. **źródło pierwotne** — dokumentacja, specyfikacja lub kod upstream,
2. **obserwacja** — działający interfejs, log lub środowisko testowe,
3. **materiał roboczy** — wyszukiwanie, które można zamknąć po zapisaniu wniosku.

Wniosek z researchu powinien trafić do zadania wraz z adresem źródła i datą sprawdzenia. Sama otwarta karta nie jest pamięcią projektu.

2.3 Dwie aplikacje AI

Dwa narzędzia mają sens, jeśli otrzymują różne role. Przykładowy podział:

Powierzchnia	Domyślna rola
AI A	implementacja z pełnym kontekstem repozytorium
AI B	niezależny review, testy lub research
przeglądarka	weryfikacja źródeł i obserwacja działania
terminal	wykonanie, testy, pomiary i receipts

Nie kopiuj automatycznie każdego promptu do obu modeli. Podwójna odpowiedź tworzy podwójny koszt odbioru. Porównanie równoległe jest uzasadnione przy decyzji trudnej do cofnięcia, dużej niepewności albo wyraźnym pomiarze jakości.

2.4 Higiena stanowiska

- Wyłącz powiadomienia o zakończeniu odpowiedzi modelu.
- Ustal dwa lub trzy okna odbioru wyników w ciągu dnia.
- Nie trzymaj sekretów w historii terminala ani treści promptu.
- Używaj osobnego profilu przeglądarki dla pracy i researchu.
- Zamykaj powierzchnie poprzedniego zadania przed rozpoczęciem następnego.

2.5 Eksperyment

Przez tydzień zrób zdjęcie lub tekstowy snapshot stanowiska na początku każdej sesji. Zapisz liczbę widocznych powierzchni niezwiązanych z aktywnym zadaniem i liczbę przerw wywołanych przez narzędzia. Następnie usuń jedną klasę przerw. Nie zmieniaj jednocześnie sprzętu, modelu i procesu — nie będziesz wiedzieć, co wpłynęło na wynik.

3. Kolejka zamiast chaosu

Każde zadanie przed wysłaniem do modelu ma kryteria akceptacji, limit czasu, limit kosztu i jasny punkt eskalacji. Wyniki nie przerywają skupienia — czekają na zaplanowany odbiór.

Kolejka jest kontraktem między generowaniem a ludzką decyzją. Bez niej każde narzędzie walczy o uwagę, a kolejność pracy ustala moment pojawienia się powiadomienia.

3.1 Minimalna karta zadania

```
task:
  id: api-validation-001
  outcome: "Nieprawidłowe wejście zwraca stabilny błąd domenowy"
  context:
    - "ścieżka modułu"
    - "obowiązujący kontrakt API"
  acceptance:
    - "test negatywny przechodzi"
    - "brak zmiany zachowania poprawnych żądań"
  budget:
    time_minutes: 30
    ai_runs: 2
  escalation:
    after_failed_attempts: 2
  owner: "human:programmer"
```

Karta nie musi być długa. Musi pozwolić odróżnić wynik użyteczny od przekonująco brzmiącej odpowiedzi.

3.2 Stany kolejki

Wystarcza sześć stanów:

1. **ready** — zadanie ma kontekst i kryteria,
2. **running** — wykonawca pracuje,
3. **review** — wynik czeka na zaplanowany odbiór,
4. **needs_input** — brakuje konkretnej decyzji albo danych,
5. **accepted** — kryteria i dowód są spełnione,
6. **discarded** — wynik odrzucono wraz z powodem.

Stan **running** nie jest postępek samym w sobie. Postęp następuje wtedy, gdy zadanie przechodzi do stanu bliższego decyzji i pozostawia nową informację.

3.3 Zasady odbioru

- Odbieraj wyniki partiami, nie natychmiast.
- Najpierw sprawdzaj zadania blokujące aktywną pracę.
- Nie uruchamiaj kolejnej iteracji bez zapisania przyczyny odrzucenia.
- Wynik bez diffu, testu lub wskazanego źródła wraca do wykonawcy.
- Po przekroczeniu limitu prób eskaluj do człowieka albo zmień strategię.

3.4 Backpressure

Jeżeli kolejka **review** osiąga ustalony limit, nie uruchamiaj następnych zadań AI. To nie jest marnowanie dostępnego modelu. To ochrona najrzadszego zasobu: zdolności człowieka do odpowiedzialnego odbioru.

3.5 Eksperyment

Ustaw limit dwóch wyników oczekujących na review. Przez pięć dni zapisuj największą długość kolejki, średni czas oczekiwania i liczbę wyników odrzuconych bez kolejnej iteracji. Jeśli kolejka często jest pełna, zmniejsz liczbę równoległych zadań albo popraw ich kryteria wejścia.

4. Dwa modele, różne role

Model A przygotowuje rozwiązanie. Model B otrzymuje diff i kryteria, a następnie szuka luk, projektuje testy lub podważa założenia. Dostawca jest bindingiem do capability, nie częścią strategii.

Obserwacje programistów korzystających z generatorów kodu rozróżniają tryb przyspieszenia, gdy człowiek zna następny krok, oraz eksploracji, gdy dopiero szuka rozwiązania [3]. Rozdzielenie ról modeli powinno wynikać z tych odmiennych potrzeb, a nie z samej dostępności kolejnego okna czatu.

Posiadanie dwóch dostępów nie jest jeszcze redundancją. Redundancja powstaje dopiero wtedy, gdy drugi tor potrafi przejąć potrzebną capability albo niezależnie wykryć błąd pierwszego.

4.1 Podział odpowiedzialności

Najprostszy wzorzec wygląda tak:

Rola	Otrzymuje	Zwraca
wykonawca	zadanie, kontekst, kryteria	mały diff i wyniki kontroli
recenzent	kryteria, diff, istotne ograniczenia	findings, testy i ocenę ryzyka
człowiek	oba wyniki i dowody	decyzję: zaakceptuj, popraw lub odrzuć

Recenzent nie powinien otrzymywać całego toku rozumowania wykonawcy. Potrzebuje artefaktu, celu i ograniczeń. Dzięki temu ma szansę zakwestionować założenie, zamiast tylko kontynuować pierwszą narrację.

4.2 Kiedy drugi model pomaga

- zmiana dotyczy bezpieczeństwa, danych albo trudnego rollbacku,
- testy są niepełne i potrzebna jest druga perspektywa,
- jeden dostawca nie ma wymaganej capability,
- wynik researchu wymaga potwierdzenia źródeł,
- awaria lub limit jednego dostawcy blokuje pracę.

4.3 Kiedy drugi model szkodzi

- zadanie jest małe, deterministyczne i dobrze pokryte testami,
- człowiek musi przeczytać dwie długie, podobne odpowiedzi,
- modele wzajemnie recenzują prozę zamiast sprawdzać artefakt,
- koszt porównania jest większy niż ryzyko zadania,
- drugi model dostaje te same błędne założenia i nie ma niezależnego dowodu.

4.4 Routing po capability

Strategia powinna mówić „implementacja”, „research”, „test design” albo „niezależny review”, a nie zawierać nazwę produktu. Osobny binding może brać pod uwagę:

- dostęp do repozytorium i narzędzi,
- długość oraz rodzaj kontekstu,
- prywatność danych,
- aktualny limit i przewidywany koszt,
- obserwowaną jakość w podobnych zadaniach.

Zmiana dostawcy nie powinna zmieniać definicji sukcesu.

4.5 Eksperyment

Wybierz dziesięć podobnych zmian. Pięć wykonaj w torze pojedynczym, a pięć w torze wykonawca-recenzent. Zmierz aktywny czas człowieka, liczbę poprawek, defekty wykryte przed akceptacją i koszt. Dwa modele są usprawnieniem tylko wtedy, gdy wartość wcześniejszego wykrycia błędów przewyższa dodatkowy odbiór.

5. Równoległość bez multitaskingu

AI może wykonywać kilka niezależnych operacji. Człowiek nie powinien odbierać ich w losowej kolejności. Drugi projekt uruchamiamy dopiero wtedy, gdy pierwszy czeka na rzeczywisty bloker zewnętrzny.

Równoległość dotyczy wykonania. Multitasking dotyczy uwagi. To rozróżnienie pozwala korzystać z przepustowości narzędzi bez utrzymywania kilku problemów w pamięci człowieka.

5.1 Pipeline zamiast wielu otwartych frontów

```
człowiek: definiuje A
AI:       wykonuje A
człowiek: definiuje B
AI:       wykonuje B
człowiek: odbiera A
testy:    weryfikują A
człowiek: podejmuje decyzję A
```

Przełączenie jest tanie tylko wtedy, gdy poprzedni kontekst został zapisany. Minimalna notatka przekazania powinna zawierać:

- aktualny cel,
- ostatnią potwierdzoną obserwację,
- otwarte pytanie,
- następne polecenie lub decyzję,
- odnośniki do diffu, testu i zadania.

5.2 Dopuszczalny drugi projekt

Drugi projekt można otworzyć, gdy pierwszy:

- czeka na zewnętrzną odpowiedź,
- wykonuje długi, automatyczny proces,
- ma zapisany stan i jednoznaczny punkt wznowienia,
- nie może zostać przyspieszony kolejną decyzją człowieka.

„Model jeszcze pisze” zwykle nie jest wystarczającym powodem. Ten czas można wykorzystać na doprecyzowanie testu, przygotowanie odbioru albo przerwę.

5.3 Limity pracy w toku

Punkt startowy dla jednej osoby:

Typ pracy	Limit
aktywne zadanie człowieka	1
zadania wykonywane przez AI	2
wyniki oczekujące na review	2
projekty wymagające bieżącego kontekstu	1

To hipoteza, nie prawo natury. Limit zwiększamy dopiero wtedy, gdy pomiary pokazują pustą kolejkę odbioru, mały rework i brak wzrostu defektów.

5.4 Eksperyment

Porównaj dwa tygodnie. W pierwszym pozwól sobie na zwyczajową liczbę otwartych zadań. W drugim zastosuj powyższe limity i obowiązkową notatkę przekazania. Zmierz lead time, liczbę wznowień, czas potrzebny na odzyskanie kontekstu oraz liczbę zadań pozostawionych bez decyzji.

II

II. Warsztat i pomiar

6	Shell jako centrum operacyjne	23
6.1	Izolacja zadania	23
6.2	Kontrakt poleceń	23
6.3	Sesja i receipt	23
6.4	Bezpieczeństwo	24
6.5	Eksperyment	24
7	Jak mierzyć efektywność	25
7.1	Minimalny zestaw	25
7.2	Jednostka porównania	25
7.3	Metryki ostrzegawcze	25
7.4	Czego nie nagradzać	26
7.5	Karta wyniku	26
8	Budżet i ograniczenia AI	27
8.1	Cztery budżety	27
8.2	Budżet zadania	27
8.3	Dwa dostępy jako odporność	27
8.4	Plan degradacji	27
8.5	Eksperyment	28

6. Shell jako centrum operacyjne

Każde równoległe zadanie dostaje osobny worktree, sesję i zestaw kontroli. Powtarzalne polecenia stają się skryptami, a wynik pracy trafia do jednej kolejki odbioru.

Terminal jest dobrym centrum operacyjnym, ponieważ pozostawia ślad i daje się automatyzować. Nie oznacza to, że wszystkie narzędzia muszą działać w shellu. Oznacza, że shell potrafi odpowiedzieć: na czym pracujemy, jaki jest stan, które kontrole przeszły i gdzie znajduje się wynik.

6.1 Izolacja zadania

Osobny `git worktree` pozwala oddzielić diff, zależności i procesy:

```
git fetch origin
git worktree add ../work/api-validation -b work/api-validation origin/main
cd ../work/api-validation
```

Nie twórz worktree bez właściciela i zadania. Nazwa katalogu, gałęzi oraz rekordu kolejki powinna prowadzić do tego samego identyfikatora.

Przed usunięciem worktree sprawdź status, wypchnij potrzebną gałąź i zachowaj receipt testów. Automatyzacja nie usprawiedliwia utraty nieodebranej pracy.

6.2 Kontrakt poleceń

Projekt powinien mieć mały, przewidywalny zestaw komend:

```
install    - odtwarza zależności z lockfile
check      - szybkie kontrole przed przekazaniem
test       - pełna weryfikacja
build      - tworzy artefakt
preview    - uruchamia bezpieczny podgląd
```

Dokładna implementacja może korzystać z `npm`, `make`, `just`, `task` albo skryptów repozytorium. Ważniejsza od nazwy narzędzia jest stabilna semantyka. Agent nie powinien zgadywać, czy `test.sh` publikuje artefakt albo zmienia dane.

6.3 Sesja i receipt

Dla każdego zadania zapisuj:

```
run:
  task_id: api-validation-001
  worktree: ../work/api-validation
  base_revision: abc123
  commands:
    - npm test
    - npm run build
  result:
    tests: passed
    build: passed
  output_revision: def456
```

To nie musi być rozbudowany system telemetryczny. Wystarczy deterministyczna odpowiedź na pytanie, co sprawdzono przed review.

6.4 Bezpieczeństwo

- Nie przekazuj modelowi plików `.env`, kluczy ani pełnych logów z tokenami.
- Preferuj referencje do sekretów zamiast ich wartości.
- Oddziel komendy odczytu od mutacji i publikacji.
- Przed publikacją wymagaj jawnego celu, rewizji i potwierdzenia testów.
- Skrypt awaryjny nie może raportować sukcesu, jeśli pominął rzeczywistą operację.

6.5 Eksperyment

Wybierz jeden często powtarzany rodzaj zadania. Przygotuj komendę `check`, szablon receptu i osobny worktree. Przez tydzień zmierz liczbę ręcznych kroków, nieudanych przekazania oraz przypadków, w których review musiało wrócić po brakujący test.

7. Jak mierzyć efektywność

Najważniejsze metryki to lead time, aktywny czas człowieka, czas oczekiwania, rework, defekty po odbiorze oraz koszt zaakceptowanej zmiany. Liczba linii kodu nie jest miarą wartości.

Kontrolowany eksperyment z GitHub Copilot wykazał istotne skrócenie czasu wykonania konkretnego zadania typu greenfield [4]. Nie należy jednak przenosić tego wyniku automatycznie na utrzymanie dojrzałego repozytorium: benchmark SWE-bench pokazuje, że rzeczywiste zgłoszenia wymagają nawigacji po kodzie, uruchamiania testów i koordynacji zmian w wielu plikach [5].

Pomiar ma wspierać decyzję o procesie, a nie tworzyć ranking programistów. Metryka bez pytania prowadzi do optymalizacji liczby, często kosztem całego systemu.

7.1 Minimalny zestaw

Metryka	Początek	Koniec	Pytanie
lead time	zadanie gotowe	wynik zaakceptowany	Jak szybko dostarczamy wartość?
czas człowieka	aktywne skupienie	odłożenie kontekstu	Ile rzadkiego zasobu zużyliśmy?
czas AI	zlecenie	wynik gotowy	Czy wykonanie jest faktycznym blokerem?
czas oczekiwania	wynik gotowy	rozpoczęcie review	Czy kolejka przekracza odbiór?
rework	pierwsze odrzucenie	zaakceptowana poprawka	Ile pracy generuje niepełny wynik?
defekty po odbiorze	akceptacja	koniec okna obserwacji	Czy przyspieszenie ukrywa koszt jakości?
koszt zmiany	pierwsze zlecenie	akceptacja	Ile kosztował cały zaakceptowany rezultat?

Koszt zmiany obejmuje nie tylko opłatę za model. Obejmuje także aktywny czas człowieka, ponowne uruchomienia, review i naprawę regresji.

7.2 Jednostka porównania

Porównuj podobne klasy zadań: mały błąd z małym błędem, migrację z migracją. Średnia ze wszystkich zmian miesza różne ryzyko i może ukryć rzeczywisty efekt.

Każdy eksperyment powinien mieć:

- stan bazowy,
- jedną badaną zmianę,
- ograniczony czas,
- miarę jakości,
- warunek zatrzymania,
- zapis odrzuconych wyników.

7.3 Metryki ostrzegawcze

Wzrost jednej z tych wartości powinien zatrzymać skalowanie:

- wyniki AI oczekujące dłużej niż samo wykonanie,

- udział reworku przekraczający pierwszą implementację,
- malejący odsetek wyników zaakceptowanych bez istotnej poprawki,
- wzrost defektów po wdrożeniu,
- rosnący koszt na zaakceptowaną zmianę mimo większej liczby uruchomień.

7.4 Czego nie nagradzać

- liczby promptów,
- tokenów zużytych w ciągu dnia,
- liczby równoległych agentów,
- linii wygenerowanego kodu,
- liczby rozpoczętych ticketów,
- zielonej odpowiedzi modelu bez niezależnego testu.

To wskaźniki aktywności, nie wartości.

7.5 Karta wyniku

```
result:
  task_class: small_bug
  accepted: true
  lead_time_minutes: 74
  human_active_minutes: 31
  ai_runtime_minutes: 12
  review_wait_minutes: 18
  rework_minutes: 9
  escaped_defects: 0
  provider_cost: 1.40
```

Po kilku porównywalnych zadaniach można sprawdzić, czy dana strategia rzeczywiście skraca lead time bez przerzucania kosztu na review i utrzymanie.

8. Budżet i ograniczenia AI

Limit zapytań, tokenów lub czasu jest częścią architektury procesu. Routing providerów powinien uwzględniać dostępność, koszt, prywatność i jakość konkretnej capability.

Nieograniczony eksperyment łatwo zamienia się w serię kolejnych prób bez nowej informacji. Budżet wymusza moment decyzji: poprawić kontekst, zmienić strategię, eskalować do człowieka albo odrzucić zadanie.

8.1 Cztery budżety

1. **Pieniądze** — koszt dostawcy, infrastruktury i narzędzi.
2. **Czas** — czas wykonania oraz maksymalny termin oczekiwania.
3. **Uwaga** — liczba wyników, które człowiek będzie musiał odebrać.
4. **Ryzyko** — dopuszczalny zakres danych, mutacji i skutków błędu.

Najtańsza odpowiedź nie musi dawać najtańszej zaakceptowanej zmiany. Jeżeli wymaga długiego review i kilku poprawek, koszt całego procesu rośnie.

8.2 Budżet zadania

```
budget:
  provider_cost_max: 3.00
  attempts_max: 2
  elapsed_minutes_max: 45
  human_review_minutes_max: 20
  data_class: internal
  mutation: proposal_only
```

Po wyczerpaniu budżetu wynik przechodzi do `needs_input`. Nie uruchamiamy automatycznie większego modelu ani kolejnej subskrypcji, jeżeli nie zmieniły się fakty lub strategia.

8.3 Dwa dostępy jako odporność

Dwóch dostawców może:

- ograniczyć wpływ awarii i limitów,
- zapewnić różne capabilities,
- umożliwić niezależny review,
- dać punkt odniesienia dla jakości i kosztu.

Nie gwarantuje jednak ciągłości. Oba narzędzia mogą zależeć od podobnych upstreamów, mieć inne zasady danych albo nie obsługiwać tego samego workflow. Przed zakupem drugiego dostępu sprawdź, którą konkretną awarię lub lukę ma pokrywać.

8.4 Plan degradacji

Proces powinien przewidywać:

Sytuacja	Reakcja
limit modelu wykonawczego	zachowaj zadanie i przełącz zgodny binding
brak modelu recenzującego	użyj testów deterministycznych i oznacz brak niezależnego review

Sytuacja	Reakcja
brak dostępu do sieci	pracuj na zapisanych źródłach albo zatrzymaj re-search
przekroczony koszt	eskaluj; nie zwiększaj budżetu bez decyzji
niezgodność danych	użyj lokalnego narzędzia albo zredagowanego kontekstu

Degradacja ma być jawna w recepcie. Wynik o słabszym dowodzie nie może wyglądać tak samo jak wynik pełnego procesu.

8.5 Eksperyment

Przez dwa tygodnie przypisuj zadaniom budżet przed uruchomieniem. Zapisuj, ile zadań przekroczyło limit oraz dlaczego. Jeżeli najczęstszą przyczyną jest niepełny kontekst, kolejny zakup nie rozwiąże problemu. Jeżeli przyczyną są rzeczywiste limity albo brak capability, drugi dostawca może mieć mierzalną wartość.

III

III. System uczący się

9	Digital Twin Portrait, router i orchestrator ticketów	31
9.1	Intencja skrócona do kontraktu	31
9.2	Łańcuch wykonania	31
9.3	Dowód z próby wdrożeniowej	32
9.4	Eksperyment dla własnego workflow	32
9.5	Wniosek	32
10	DSL: idea, którą można uruchomić	33
10.1	Języki i odpowiedzialności	33
10.2	Kompresja bez utraty znaczenia	34
10.3	Minimalny przebieg	34
10.4	Process Pack	34
10.5	Reguła ewolucji	35
10.6	Granica między przykładem a runtime	35
11	Cotygodniowa retrospektywa	37
11.1	Dane wejściowe	37
11.2	Szablon	37
11.3	Cztery pytania	37
11.4	Pierwsze trzydzieści dni	38

9. Digital Twin Portrait, router i orchestrator ticketów

Digital Twin jest portretem sposobu pracy aktora: specjalizacji, kryteriów jakości, rytmu, kosztu i aktualnego obciążenia. Portret jest **tylko do odczytu**. Może dostarczyć sygnał routingowy, ale nie przyznaje authority i nie może autonomicznie zmienić własnych praw.

W tej książce używamy nazwy **DOQL/Situation** dla agregacji sytuacji Digital Twin. Odróżniamy ją od spotykanej w projektach deklaracji `app.doql.less`, która opisuje strukturę aplikacji i jej środowisko.

Rozdzielenie obserwacji od uprawnień jest ważniejsze niż trafność samego profilu:

- DOQL opisuje sytuację i portret,
- router wybiera kandydata dopiero po pokryciu AQL i URI,
- AQL określa granice działania,
- OQL materializuje przebieg,
- EQL dopuszcza handoff,
- receipt dowodzi, co faktycznie wykonano.

9.1 Intencja skrócona do kontraktu

Opis narracyjny:

Router może korzystać z portretu Digital Twin przy wyborze człowieka, bota albo serwisu, ale nie może na jego podstawie rozszerzyć uprawnień. Kandydat musi mieć pokrycie AQL i rozwiązywalny URI. Następny ticket może rozpocząć się dopiero po terminalnym receipcie poprzednika i zielonym EQL.

Ta sama decyzja w krótkim DSL:

```
intent:
  goal: route_ticket
  portrait: read_only
  authority: inherited_only

requires:
  aql_coverage: true
  uri_resolvable: true
  eql_handoff: green
```

Proza wyjaśnia sens. DSL oddziela pola, które można walidować, testować, wersjonować i wdrażać.

9.2 Łańcuch wykonania

Orchestrator materializuje `subactor.ticket-execution-chain/v1`. Kandydatem może być człowiek, bot lub serwis. Handoff wymaga terminalnego receiptu poprzednika i zielonego EQL:

```
Intent -> Composer -> DOQL snapshot -> Digital Twin Router
                        /      |      \
                    human  bot   service
                        \      |      /
                    receipt + EQL handoff
                        |
                    następny ticket
```

Odmowa readiness nie może tworzyć pętli bez postępu. False-ready przechodzi do `waiting_input` z dokładnym kodem bramki. Scheduler nie ponawia niezmienionego błędu.

```
on_false_ready:
  state: waiting_input
  gate_code: required
  retry_unchanged: false
```

9.3 Dowód z próby wdrożeniowej

Poniższe liczby są snapshotem pojedynczej próby projektu Subactor z 24 lipca 2026, zapisanym w dołączonym Process Packu. Nie są benchmarkiem wydajności ani podstawą do uogólnienia na inne systemy. Pokazują format dowodu:

- przeskanował 55 ticketów,
- rozważył 33 kandydatów,
- sprawdził pięć oczekujących ticketów,
- nie promował żadnego bez zielonych bramek,
- zakończył `queue_execution` w 18,8 s.

Kontroler wykonuje sweep co 300 000 ms. Błąd wymagający reakcji może zlecić dodatkowy cykl po 1 s, z deduplikacją i cooldownem 5 minut. Sam błąd nie rozszerza authority ani nie omija lifecycle.

W rejestrze lifecycle `digital_twin_profile` miał status `partial`, a `autonomy.enabled=false`. Portret był użyteczny dla routingu, ale nie otrzymał prawa do samodzielnej migracji profilu lub modyfikacji uprawnień.

Próba potwierdziła wykonanie wewnętrznego lejka przez `operations-operator-bot`, audyt connectorów social przez `marketing-operator-bot` oraz powiązanie dwóch skrzynek Plesk z portretami w Organization Core. Research PLF-847 pozostał poprawnie zablokowany do czasu implementacji dokładnych adapterów browser i analytics.

9.4 Eksperyment dla własnego workflow

Nie zaczynaj od profilu osobowości ani oceny „który model jest najlepszy”. Zbuduj mały portret operacyjny z faktów:

```
portrait:
  observed_at: "2026-07-24T09:00:00Z"
  task_classes:
    small_bug:
      accepted_changes: 8
      median_human_minutes: 24
      review_required: true
  current:
    human_active_tasks: 1
    ai_running_tasks: 2
    waiting_reviews: 1
  autonomy:
    enabled: false
```

Router może użyć tych danych do rekomendacji wykonawcy albo kolejności odbioru. Nie może na ich podstawie sam nadać dostępu do repozytorium, zwiększyć budżetu czy opublikować zmiany.

9.5 Wniosek

Wartością Digital Twina nie jest naśladowanie człowieka. Jest nią coraz lepszy, sprawdzalny routing pracy bez rozszerzania uprawnień na podstawie predykcji. Dobry portret mówi również, czego jeszcze nie wiadomo i kiedy obserwacja przestaje być świeża.

10. DSL: idea, którą można uruchomić

Naturalny język dobrze przenosi kontekst, motywację i wyjątki. Słabo nadaje się jednak do jednoznacznego wykrywania brakujących pól, porównywania wersji i automatycznego testowania.

Badania ReAct pokazują wartość przeplatania rozumowania z działaniami wykonywanymi w środowisku [6]. Z kolei SWE-agent wskazuje, że projekt interfejsu agent-komputer wpływa na skuteczność agenta [7]. DSL w tej książce pełni właśnie rolę ograniczonego interfejsu: zmniejsza przestrzeń możliwych działań i wymusza jawny dowód.

DSL zwiększa wartość idei, gdy formalizuje pięć elementów:

1. intencję,
2. granice authority,
3. przebieg operacji,
4. oczekiwany dowód,
5. reakcję na brak gotowości.

10.1 Języki i odpowiedzialności

W przykładach używamy rodziny DSL-i rozwijanych w projektach Subactora:

Warstwa	Rola w eksperymencie
Intent Contract	zaakceptowany cel, ograniczenia i Definition of Done
Strategy DSL	wybór zatwierdzonej strategii oraz abstrakcyjnych capabilities
AQL	authority aktora, allow/deny, limity i delegacja
OQL	plan operacji oraz zależności między krokami
URI Process	binding capability do konkretnego runtime lub connectora
EQL/Expectation	preflight, postcondition i read-back wyniku
EQL/Pressure	ograniczona ocena presji, kosztu lub momentu eskalacji
TestQL	deterministyczna bramka gotowości i jakości
DOQL/Situation	ograniczony obraz sytuacji Digital Twin
DQL/Diagnostic	diagnostyka inwariantów tylko do odczytu
SODL	append-only historia zdarzeń i decyzji

Nie są to wymienne nazwy dla jednego pliku. Każdy język chroni inną granicę. AQL nie wykonuje operacji, DOQL nie nadaje uprawnień, a zielona odpowiedź connectora nie zastępuje EQL.

10.1.1 Kwalifikuj profile

W ekosystemie występują historyczne lub projektowe profile o tych samych skrótach. Dlatego w książce piszemy:

- **DOQL/App** dla deklaracji aplikacji w składni `app.doql.less`,
- **DOQL/Situation** dla agregacji sytuacji Digital Twin,
- **DQL/Diagnostic** dla read-only autodiagnostyki,
- **DQL/DOM** dla ograniczonego planu adaptacji interfejsu,

- EQL/Expectation, EQL/Pressure i EQL/Web dla różnych profili EQL.

Kwalifikator jest częścią kontraktu czytelniczego. Zapobiega przypadkowemu przeniesieniu semantyki mutacji do języka diagnostycznego.

10.2 Kompresja bez utraty znaczenia

Krótszy zapis nie jest celem samym w sobie. Dobra kompresja usuwa powtórzenia, ale zachowuje decyzje:

```
intent:
  id: route-ticket-v1
  goal: "Wybrać bezpiecznego wykonawcę ticketu"

constraints:
  portrait_mutation: forbidden
  authority_expansion: forbidden

definition_of_done:
  - terminal_receipt
  - eql_handoff_green
```

Taki kontrakt może jednocześnie:

- przejść walidację schematu,
- zostać porównany jako mały diff,
- zasilić router,
- wygenerować test gotowości,
- wejść do dokumentacji jako przykład.

10.3 Minimalny przebieg

```
idea
-> Intent Contract
-> AQL + URI validation
-> OQL prototype
-> EQL + TestQL gate
-> SODL receipt
-> retrospective
-> version + 1
```

Każdy przebieg dostarcza dowodu. Następna wersja zmienia jedną hipotezę lub regułę, dzięki czemu można przypisać zmianę wyniku do konkretnego diffu.

10.4 Process Pack

```
digital-twin-router/
├─ intent.yaml
├─ authority.contract.aql
├─ routing.strategy.yaml
├─ portrait.doql.json
├─ execution-chain.oql
├─ handoff.eql
├─ readiness.testql
├─ events.sodl.jsonl
└─ retrospective.md
```

Pakiet z książki jest demonstracją edukacyjną opartą na rozwiązaniu opisanym w Subactorze. Kanoniczna składnia produkcyjna pozostaje w repozytorium projektu, a przykład może ewoluować niezależnie wraz z kolejnymi wydaniem książki.

10.5 Reguła ewolucji

Nie zwiększamy autonomii dlatego, że nowa wersja brzmi przekonująco. Zwiększamy ją, gdy walidowalny kontrakt, testy, receipts i wyniki powtarzalnych prób potwierdzają bezpieczny postęp.

10.6 Granica między przykładem a runtime

Process Pack dołączony do książki jest materiałem edukacyjnym. Składnia inspirowana Subactorem nie staje się przez to produkcyjnym grantem. Przed wykonaniem system musi:

1. zwalidować dokument względem konkretnej wersji schematu,
2. rozwiązać principal i authority z rzeczywistego kontraktu,
3. związać plan, payload i approval jednym hashem,
4. wybrać istniejący URI przez kontrolowany binding,
5. zapisać niezależny receipt rezultatu.

LLM może zaproponować treść kontraktu, ale nie powinien sam tworzyć authority, sekretu, transportu ani nieznanego URI.

11. Cotygodniowa retrospektywa

Raz w tygodniu wybieramy jedną rzecz do zachowania, jedną do usunięcia, jedną do automatyzacji i jeden eksperyment. Proces ma być prostszy po każdym cyklu.

Retrospektywa nie jest spotkaniem do wymiany wrażeń. Jest domknięciem pętli: porównuje intencję z wykonaniem, wskazuje dowód i tworzy małą zmianę następnej wersji procesu.

11.1 Dane wejściowe

Przed rozpoczęciem zbierz:

- zaakceptowane i odrzucone zadania,
- czas aktywnej pracy człowieka,
- długość kolejki review,
- przekroczone budżety,
- rework i defekty po odbiorze,
- zdarzenia, przy których potrzebna była eskalacja.

Nie potrzebujesz kompletnej telemetrii. Potrzebujesz danych wystarczających do odróżnienia zdarzenia jednorazowego od powtarzającego się wzorca.

11.2 Szablon

```
retrospective:
  period: 2026-W31

  expected:
    hypothesis: "Niezależny review zmniejszy rework"
    success: "rework_minutes -20%, bez wzrostu lead_time"

  observed:
    comparable_tasks: 8
    result: inconclusive
    evidence:
      - "review wykrył dwa brakujące testy"
      - "kolejka odbioru dwukrotnie przekroczyła limit"

  decisions:
    keep: "niezależny review dla zmian wysokiego ryzyka"
    remove: "podwójne odpowiedzi dla zmian mechanicznych"
    automate: "zapis czasu gotowości wyniku"
    experiment: "odbiór wyników o 11:30 i 15:30"

  owner: "human:programmer"
  review_after: "7d"
```

11.3 Cztery pytania

1. **Co zachować?** Wskaż praktykę i dowód, nie ogólne odczucie.
2. **Co usunąć?** Usuń krok, który nie zmienia decyzji ani jakości.
3. **Co zautomatyzować?** Wybierz powtarzalną czynność z jednoznacznym wynikiem.
4. **Co sprawdzić?** Zmień jedną hipotezę i określ warunek zatrzymania.

Jeżeli wynik jest niejednoznaczny, nie wymuszaj werdyktu. Zapisz brakujące dane i zaprojektuj mniejszą próbę.

11.4 Pierwsze trzydzieści dni

- **Tydzień 1:** tylko pomiar stanu bazowego i Human WIP = 1.
- **Tydzień 2:** kolejka odbioru oraz stałe okna review.
- **Tydzień 3:** rozdzielenie ról dwóch modeli dla jednej klasy zadań.
- **Tydzień 4:** budżety, plan degradacji i pierwszy Process Pack.

Po miesiącu powinienes mieć mniej zasad niż na początku, ale każda z nich powinna być oparta na obserwacji. Celem nie jest maksymalna autonomia. Celem jest system pracy, który potrafi wyjaśnić, dlaczego dany poziom autonomii jest w tej chwili bezpieczny i opłacalny.

IV

IV. Architektura wykonywalna

12	Od SOA do POA: URI Process	41
12.1	URI jest nazwą, nie transportem	41
12.2	Kontrakt procesu	42
12.3	Od bindings do dowodu	42
12.4	Przykład publikacji	42
12.5	Granice bezpieczeństwa	43
12.6	Eksperyment	43
13	Książka jako wykonywalny system publikacji	45
13.1	Jedno źródło, różne projekcje	45
13.2	Audio jako artefakt	45
13.3	Reprodukowalność	46
13.4	Test wydania	46
14	Forma współpracy, authority i dowód pracy	47
14.1	Główne formy współpracy	47
14.2	Lifecycle relacji	48
14.3	Ile może organizować pracodawca	48
14.4	Proces zamiast totalnego monitoringu	49
14.5	Granica między UoP a relacją cywilną	50
14.6	Wniosek dla modelu actor/agent	50

12. Od SOA do POA: URI Process

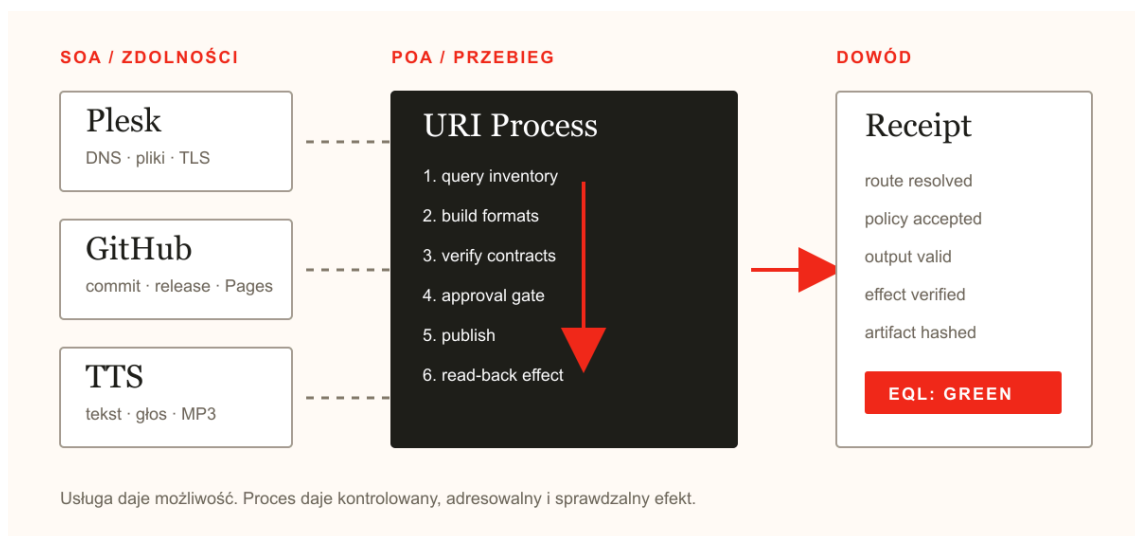
Architektura zorientowana na usługi odpowiada na pytanie, **jaka usługa udostępnia zdolność**. Architektura zorientowana na proces odpowiada na pytanie, **jaki kontrolowany przebieg doprowadzi do oczekiwanego efektu**.

W tej książce używamy skrótu **POA** jako opisu architektury zorientowanej na proces. Nie jest to oficjalna nazwa standardu `urirun`, lecz model pomagający zrozumieć jego kontrakty.

Wymiar	SOA	POA
jednostka	usługa	proces lub krok procesu
interfejs	endpoint API	typowany kontrakt wykonania
kompozycja	wywołania usług	plan, zależności i bramki
sukces	odpowiedź usługi	zweryfikowany efekt
lokalizacja	związana z wdrożeniem	ustalana przez routing

POA nie zastępuje SOA. Długo działająca usługa nadal posiada cykl życia, port, stan i endpointy. Proces składa zdolności jednej lub wielu usług w operację, którą można zaplanować, autoryzować, wykonać i zweryfikować.

Badania nad wzorcami workflow pokazują, że kontrola przepływu, danych, zasobów i wyjątków jest osobnym problemem względem implementacji pojedynczej usługi [8].



Rysunek 12.1: Relacja usługi, procesu URI, routera i dowodu wykonania.

12.1 URI jest nazwą, nie transportem

RFC 3986 rozdziela identyfikację od interakcji: URI identyfikuje zasób, lecz nie gwarantuje konkretnego sposobu dostępu [9]. To rozdzielenie jest podstawą URI Process. Adres:

```
plesk://host/site/command/sync
```

nie mówi, czy wykonanie użyje lokalnej funkcji, HTTP, SFTP, kolejki czy zdalnego node'a. Mówi, **jaki logiczny proces ma zostać wykonany**. Router wybiera rzeczywisty runtime dopiero po sprawdzeniu registry, targetu i polityki.

Podobną korzyść daje jednolity interfejs REST: implementacja może ewoluować niezależnie od udostępnianej zdolności [10]. URI Process przenosi tę zasadę z reprezentacji zasobów na wykonywalne operacje.

12.2 Kontrakt procesu

Minimalny proces zawiera:

```
{
  "id": "publish",
  "name": "Opublikuj wydanie",
  "actor": "system",
  "uri": "plesk://host/site/command/sync",
  "payload": {"source_dir": "dist"},
  "depends_on": ["verify"],
  "human_approval": true,
  "timeout_seconds": 120,
  "retries": 1
}
```

Kontrakt oddziela:

- tożsamość kroku,
- aktora,
- logiczny adres procesu,
- dane wejściowe,
- zależności,
- approval,
- zachowanie czasu i ponowień.

Lista takich obiektów tworzy mały DAG. Zależności są walidowane przed wykonaniem, a cykl jest błędem kontraktu, nie problemem odkrywanym dopiero podczas dispatchu.

12.3 Od bindings do dowodu

```
deklaracja connectora
-> bindings + schema
-> skompilowane registry
-> routing diagnosis
-> policy / grant
-> dispatch przez adapter
-> walidacja outputu
-> EQL read-back
-> journal + artifact
```

`query` oznacza operację tylko do odczytu. `command` oznacza możliwy efekt uboczny i wymaga polityki wykonania. Sama obecność słowa `command` w URI nie jest jednak wystarczającym zabezpieczeniem. Docelowo efekt powinien być również typowanym polem kontraktu, aby bramka nie opierała decyzji wyłącznie na parsowaniu tekstu adresu.

12.4 Przykład publikacji

Publikacja strony może korzystać z wielu usług, lecz pozostaje jednym procesem biznesowym:

```
repo://host/release/query/inventory
-> book://host/release/command/build
-> test://host/release/query/verify
-> plesk://host/site/command/release-upload
-> plesk://host/site/command/release-verify
-> plesk://host/site/command/release-activate
-> https://public/site/query/fingerprint
```

Odpowiedź ok: `true` z uploadu oznacza jedynie poprawny dispatch. Dopiero read-back publicznego fingerprintu jest dowodem opublikowanego efektu.

Kompletny przykład znajduje się w dołączonym pakiecie `dsl/examples/uri-process-publication/`.

12.5 Granice bezpieczeństwa

URI Process nie nadaje authority. Rozwiązywalna trasa mówi, że coś **może** zostać wykonane technicznie, a nie że dany aktor **ma prawo** to wykonać.

Dlatego proces wymaga osobnych warstw:

1. AQL ogranicza aktora i klasy efektów.
2. Router potwierdza target oraz capability.
3. Grant wiąże zgodę z hashem planu i payloadu.
4. EQL sprawdza stan po wykonaniu.
5. Journal odróżnia dispatch od efektu.

LLM może wybrać proces wyłącznie z registry. Nie powinien wymyślać URI, transportu, surowej komendy ani uprawnień.

12.6 Eksperyment

Wybierz proces, który obecnie jest pojedynczym skryptem lub runbookiem.

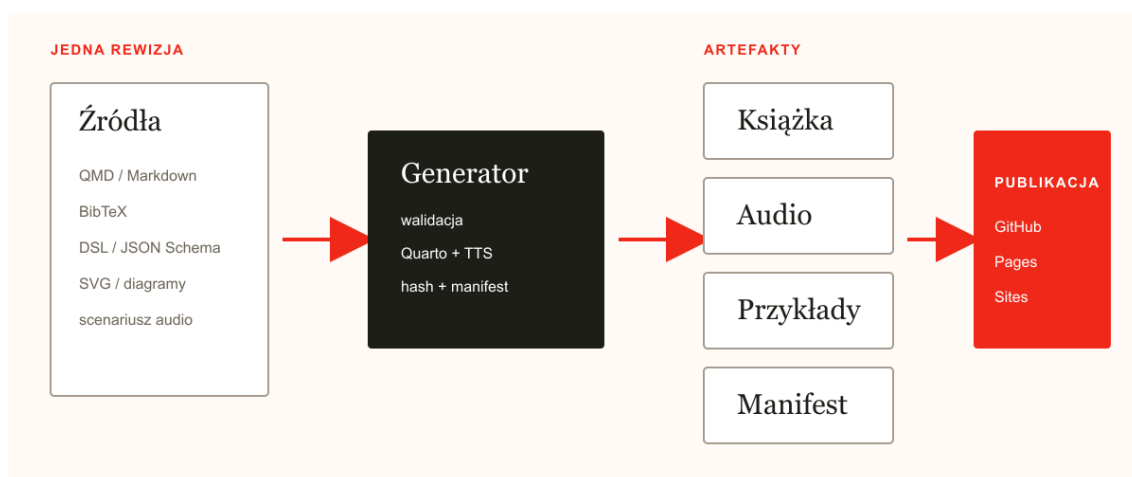
1. Rozbij go na kroki `query` i `command`.
2. Nadaj każdemu stabilny URI.
3. Zapisz zależności oraz wymagane approval.
4. Dodaj read-back efektu.
5. Uruchom najpierw w trybie dry-run.
6. Porównaj liczbę niejawnych założeń przed i po formalizacji.

Sukcesem nie jest większa liczba URI. Sukcesem jest proces, którego granice, miejsce wykonania i efekt można wyjaśnić na podstawie kontraktu i receiptu.

13. Książka jako wykonywalny system publikacji

Książka jako kod nie kończy się na przechowywaniu rozdziałów w Git. Źródło powinno deterministycznie zasilać wszystkie kanały:

```
QMD + BibTeX + DSL + SVG
-> walidacja
-> HTML + PDF + EPUB + DOCX
-> tekst narratora
-> TTS + MP3
-> manifest wydania
-> GitHub Pages + landing page
```



Rysunek 13.1: Pipeline publikacji wieloformatowej.

13.1 Jedno źródło, różne projekcje

HTML służy do wyszukiwania i linkowania. EPUB daje płynny skład na czytniku. PDF utrzuła skład. DOCX jest formatem wymiany redakcyjnej. Audio jest odrębną projekcją narracyjną, która usuwa tabele, kod i elementy czysto wizualne.

Generator nie powinien kopiować wiedzy ręcznie. Powinien:

- renderować wszystkie formaty z tej samej rewizji,
- pakować przykłady DSL,
- tworzyć manifest z rozmiarami i sumami SHA-256,
- podpisywać pochodzenie numerem wersji i commitem,
- publikować tylko po przejściu walidacji.

13.2 Audio jako artefakt

Synteza mowy jest operacją zewnętrzną i kosztową. Tekst dzielimy w granicach akapitów, a wynik każdego segmentu zapisujemy osobno. Manifest zachowuje nazwę providera, głosu, modelu, liczbę znaków i informację o głosie syntetycznym.

Provider nie jest częścią treści. Ten sam tekst może zostać przetworzony przez Google Cloud Text-to-Speech, OpenAI, ElevenLabs albo lokalny `espeak-ng`. Wydanie testowe wykorzystuje lokalny provider,

aby CI nie wymagało sekretów. Wydanie produkcyjne może użyć Google lub innego API po dostarczeniu credentials.

13.3 Reprodukowalność

Manifest wydania powinien umożliwić odpowiedź na cztery pytania:

1. Z jakiego commita powstał plik?
2. Jaki generator i provider go utworzył?
3. Czy plik zmienił się od czasu publikacji?
4. Czy wszystkie linki landing page prowadzą do tej samej wersji?

Dlatego suma kontrolna jest częścią produktu, a nie wyłącznie szczegółem CI.

13.4 Test wydania

Przed publikacją:

```
validate sources
-> render book
-> validate links and formats
-> probe audio with ffprobe
-> build application
-> smoke test landing page
-> commit and push
-> deploy saved Sites version
```

Proces publikacji jest przykładem POA opisanym w poprzednim rozdziale: łączy usługi, lecz jego jednostką sukcesu jest dostępne i zweryfikowane wydanie.

14. Forma współpracy, authority i dowód pracy

⚠ Zakres rozdziału

To materiał edukacyjny, nie porada prawna ani wzór umowy. Ocena zależy od treści kontraktu i faktycznego sposobu współpracy. Stan źródeł sprawdzono 24 lipca 2026 r.; przed zastosowaniem należy ponownie sprawdzić akty prawne i skonsultować konkretny przypadek.

Najważniejsza korekta brzmi: **umowa zlecenia nie jest umową rezultatu**. Zarówno stosunek pracy, jak i zlecenie opierają się zasadniczo na starannym działaniu. Oznaczony rezultat jest centralny przede wszystkim dla umowy o dzieło.

Nie oznacza to, że zleceniodawca nie może ustalać sposobu współpracy. Artykuł 737 Kodeksu cywilnego zakłada możliwość wskazania sposobu wykonania zlecenia, a art. 740 wymaga udzielania potrzebnych informacji o przebiegu sprawy i złożenia sprawozdania [11].

14.1 Główne formy współpracy

Forma	Co faktycznie „kupi” organizacja	Wpływ na sposób pracy	Co można kontraktować lub organizować	Główna walidacja
Umowa o pracę	pracę określonego rodzaju, wykonywaną osobicie pod kierownictwem	duży	środowisko, procedury, kolejność, raportowanie, checkpoiny, zgodne z prawem polecenia	staranność, proces, jakość i realizacja zadań
Zlecenie / usługi	staranne wykonywanie umówionych czynności	średni	zakres, instrukcje, standardy, informacje o przebiegu, dokumentację i bezpieczną infrastrukturę	należyte wykonywanie czynności zgodnie z umową
Umowa o dzieło	oznaczony, indywidualny i sprawdzalny rezultat	mały operacyjnie, duży przy odbiorze	specyfikację, format, termin, ograniczenia, testy odbiorowe i kamienie milowe	zgodność dzieła ze specyfikacją i kryteriami odbioru
B2B	usługę albo rezultat, zależnie od kontraktu	kontraktowy	SLA, standardy bezpieczeństwa, interfejs współpracy, artefakty, logi i kryteria odbioru	SLA, należyte wykonanie lub rezultat – według umowy

Forma	Co faktycznie „kupuje” organizacja	Wpływ na sposób pracy	Co można kontraktować lub organizować	Główna walidacja
Praca tymczasowa	pracę osoby zatrudnionej przez agencję	duży, ale podzielony	pracodawca użytkownik wyznacza zadania, organizuje pracę i kontroluje wykonanie	jak przy pracy, z ustawowym podziałem obowiązków
Kontrakt menedżerski	zarządzanie organizacją albo jej częścią	zależny od podstawy prawnej	mandat, cele, budżety, uprawnienia, KPI i raportowanie	należyta staranność, zgodność z mandatem i umówione wyniki

B2B nie jest jednym nazwanym typem umowy. Opisuje relację przedsiębiorców, pod którą może znajdować się świadczenie usług, dzieło, umowa agencyjna albo kontrakt mieszany. Podobnie „kontrakt menedżerski” jest nazwą praktyczną; jego charakter trzeba ustalić z konkretnej treści i sposobu wykonania. PIP podkreśla, że B2B jest relacją niezależnych przedsiębiorców, a o kwalifikacji zatrudnienia nie rozstrzyga sama etykieta dokumentu [12].

14.2 Lifecycle relacji

Forma	Typowy cykl
UoP	rola i obowiązki → onboarding → przydzielanie pracy → wykonywanie poleceń → feedback i ocena → kolejne zadania albo zakończenie
Zlecenie	zakres czynności → instrukcje i warunki → staranne działanie → informacje o przebiegu → sprawozdanie i rozliczenie → kontynuacja albo wypowiedzenie
Dzieło	specyfikacja → kryteria odbioru → samodzielne wykonanie → dostarczenie → testy → odbiór lub poprawki → zapłata
B2B	umowa ramowa → zamówienie/SOW → SLA i ograniczenia → wykonanie → dowody i milestone’y → odbiór/faktura → odnowienie albo zakończenie

Artykuł 627 Kodeksu cywilnego definiuje dzieło przez zobowiązanie do wykonania oznaczonego dzieła. Zamawiający nie musi jednak ignorować wykonania: jeżeli praca przebiega wadliwie albo sprzecznie z umową, art. 636 pozwala wezwać do zmiany sposobu wykonania i wyznaczyć odpowiedni termin [11]. Nie jest to automatycznie prawo do codziennego kierowania wykonawcą tak jak pracownikiem.

14.3 Ile może organizować pracodawca

Artykuł 22 Kodeksu pracy wiąże stosunek pracy z wykonywaniem pracy pod kierownictwem pracodawcy oraz w wyznaczonym miejscu i czasie. Artykuł 100 wymaga pracy sumiennej i starannej oraz stosowania się do zgodnych z prawem i umową poleceń dotyczących pracy [13].

Organizacja może zatem wymagać między innymi:

Obszar	Przykładowa reguła procesu
środowisko	zadania wykonujemy w firmowym repozytorium i na zarządzanym urządzeniu

Obszar	Przykładowa reguła procesu
proces	każde zadanie przechodzi plan, wykonanie, test i review
dowody	zmiana stanu ticketu wymaga wyniku testu albo artefaktu
komunikacja	blokada przekraczająca ustalony czas uruchamia eskalację
jakość	zadanie nie jest ukończone bez testów i Definition of Done
kontrola	działanie wysokiego ryzyka wymaga akceptacji drugiej osoby

Przy pracy tymczasowej pracodawca użytkownik wyznacza zadania i kontroluje ich wykonanie, a agencja pozostaje pracodawcą. Ustawa rozdziela więc obowiązki organizacyjne i formalne między dwa podmioty [14].

14.4 Proces zamiast totalnego monitoringu

Prawo organizowania pracy nie oznacza nieograniczonego śledzenia człowieka. Najbezpieczniejszy projekt systemu zapisuje **zdarzenia procesu i dowody wykonania**, a nie każdy klik, treść prywatnej komunikacji czy ciągły obraz ekranu.

UODO wskazuje, że monitoring musi realizować określony prawem cel, przejść test niezbędności i proporcjonalności oraz ustąpić mniej inwazyjnemu środkowi, jeśli ten pozwala osiągnąć ten sam cel [15].

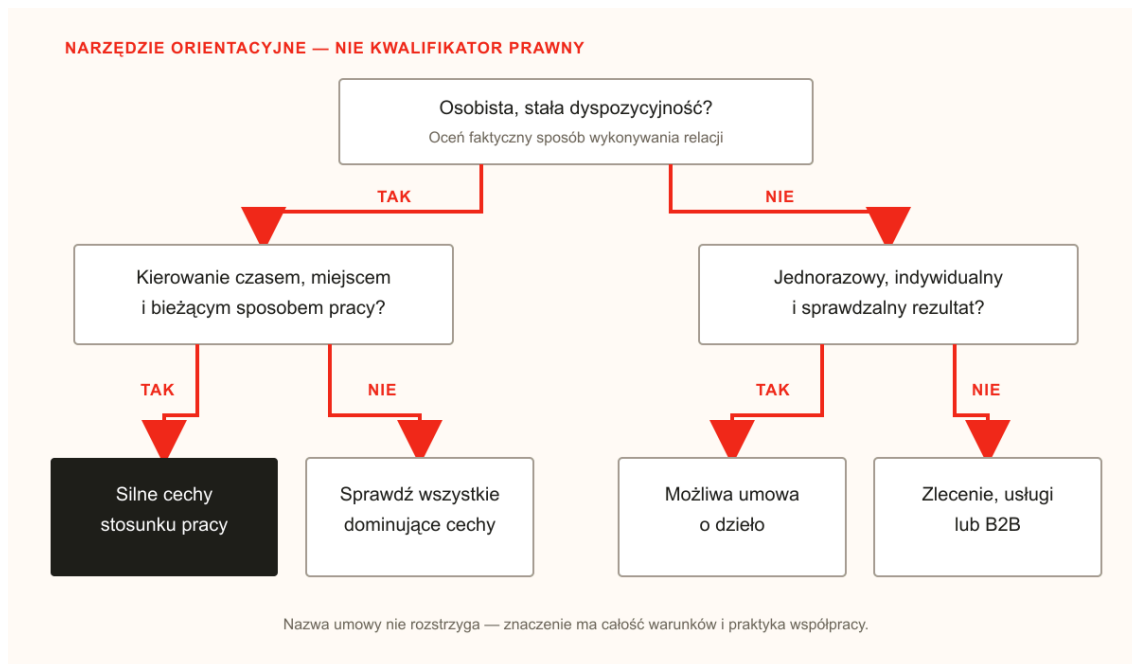
preferowane:

```
ticket.started
test.completed
approval.requested
artifact.published
```

ryzykowne bez odrębnej podstawy i analizy:

```
every.keystroke
continuous.screenshot
private.message.content
```

14.5 Granica między UoP a relacją cywilną



Rysunek 14.1: Orientacyjne drzewo cech relacji współpracy.

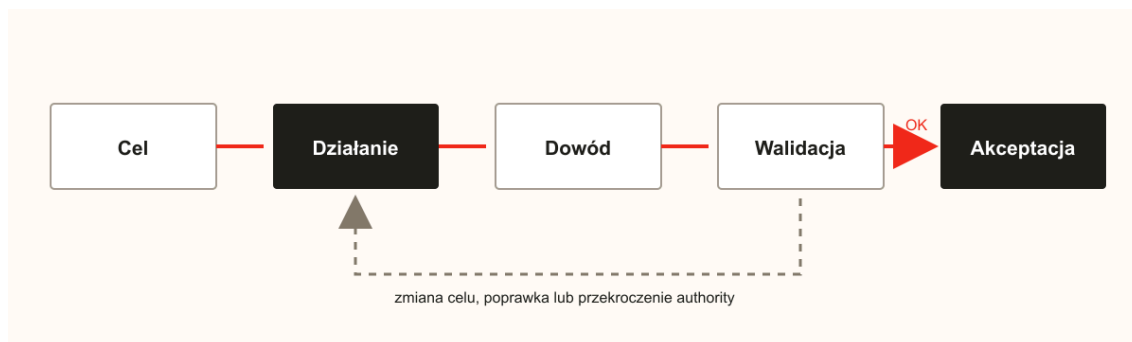
To drzewo jest narzędziem orientacyjnym, nie kwalifikatorem prawnym. Artykuł 22 Kodeksu pracy zabrania zastępowania umowy o pracę kontraktem cywilnym przy zachowaniu warunków stosunku pracy [13]. PIP wymienia podporządkowanie, osobiste świadczenie, powtarzalność, miejsce, czas i ryzyko pracodawcy jako istotne cechy, które trzeba oceniać łącznie [16].

14.6 Wniosek dla modelu actor/agent

Forma prawna wpływa na zakres authority, ale jakość wytwarza protokół pracy.

```

ticket
goal: oczekiwany stan końcowy
authority: dozwolone działania
constraints: środowisko, bezpieczeństwo, budżet
checkpoints: kiedy potrzebna jest walidacja
evidence: jakie zdarzenia i artefakty należy zapisać
acceptance: walidowalne kryteria ukończenia
escalation: kiedy actor musi oddać decyzję
  
```



Rysunek 14.2: Pętla od celu przez działanie, dowód i walidację do akceptacji.

Na UoP pracodawca może tę pętlę w dużym stopniu organizować w granicach prawa i umowy. Przy zleceniu lub B2B powinna wynikać z umówionego protokołu współpracy. Przy dziele koncentruje się przede wszystkim na specyfikacji, ryzykach, wykrywaniu wadliwego wykonania i odbiorze oznaczonego rezultatu.

Ciągła walidacja kosztuje zlecającego, ale część kosztu można przenieść do automatycznych testów. Człowiek uczestniczy wtedy głównie przy wyjątku, zmianie celu, przekroczeniu authority albo działaniu wysokiego ryzyka.



Aneksy

15	Mapa badań i siła dowodu	55
15.1	Jak interpretować wyniki	55
16	Process Pack do pobrania	57
16.1	Jak pracować z przykładem	57
16.1	Bibliografia	59

15. Mapa badań i siła dowodu

Publikacje w książce pełnią trzy role: dostarczają obserwacji o pracy ludzi, wyników eksperymentów z AI oraz modeli formalnych dla workflow i architektury. Nie wszystkie wyniki mają tę samą siłę ani zakres.

Obszar	Źródło	Co wspiera	Ograniczenie
przerwania	C. Parnin i S. Rugaber [1]	koszt rekonstrukcji kontekstu	badanie obserwacyjne
produktywność	A. N. Meyer, T. Fritz, G. C. Murphy, i T. Zimmermann [2]	wielowymiarowe rozumienie produktywności	percepcja nie jest samym wynikiem
interakcja z AI	S. Barke, M. B. James, i N. Polikarpova [3]	tryby przyspieszenia i eksploracji	20 uczestników
szybkość z AI	S. Peng, E. Kalliamvakou, P. Cihon, i M. Demirer [4]	eksperymentalny wpływ na jedno zadanie	zadanie greenfield
zadania repozytoryjne	C. E. Jimenez <i>et al.</i> [5]	trudność realnych issue	benchmark nie odwzorowuje całej pracy
interfejs agenta	J. Yang <i>et al.</i> [7]	wpływ ACI na skuteczność	wynik zależny od benchmarku
workflow	W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, i A. P. Barros [8]	wzorce kontroli przepływu	domena BPM
URI i REST	T. Berners-Lee, R. Fielding, i L. Masinter [9]; R. T. Fielding [10]	identyfikacja i jednolitość interfejs	nie definiują URI Process
granica możliwości AI	F. Dell'Acqua <i>et al.</i> [17]	nierówny wpływ AI zależnie od zadania	praca konsultantów, nie programowanie

15.1 Jak interpretować wyniki

Nie traktujemy pojedynczego procentu jako stałej właściwości AI. Wynik zależy od zadania, ludzi, narzędzia, czasu badania i kryterium jakości. Dlatego książka zaleca lokalny eksperyment porównawczy zamiast przyjęcia obcej średniej jako obiecanego zwrotu.

W szczególności szybsze ukończenie krótkiego zadania nie dowodzi mniejszego reworku, bezpieczniejszej publikacji ani lepszej pracy z dojrzałym systemem. Potwierdza hipotezę, którą należy ponownie przetestować w swojej klasie zadań.

16. Process Pack do pobrania

Repozytorium zawiera trzy kompletne przykłady:

1. `two-models` — implementacja i niezależny review przy Human WIP = 1.
2. `digital-twin-router` — routing bez rozszerzania authority.
3. `uri-process-publication` — wydanie książki jako kontrolowany URI Process.

Pakiet ZIP jest generowany przy każdym wydaniu. Zawiera źródła DSL, schematy JSON i README, ale nie zawiera sekretów ani produkcyjnych grantów.

16.1 Jak pracować z przykładem

```
skopiuuj katalog
-> zmień intent i payload
-> zwaliduj schema
-> wykonaj query/dry-run
-> sprawdź AQL
-> zatwierdź command
-> wykonaj EQL read-back
-> zapisz SODL i retrospektywę
```

Przykłady są materiałem edukacyjnym. URI należy związać z registry rzeczywistego runtime'u, a authority rozwiązać z aktywnego kontraktu organizacji.

Bibliografia

- [1] C. Parnin i S. Rugaber, „Resumption Strategies for Interrupted Programming Tasks”, *Software Quality Journal*, t. 19, nr 1, s. 5–34, 2011, doi: [10.1007/s11219-010-9104-9](https://doi.org/10.1007/s11219-010-9104-9).
- [2] A. N. Meyer, T. Fritz, G. C. Murphy, i T. Zimmermann, „Software Developers' Perceptions of Productivity”, w *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, s. 19–29. doi: [10.1145/2635868.2635892](https://doi.org/10.1145/2635868.2635892).
- [3] S. Barke, M. B. James, i N. Polikarpova, „Grounded Copilot: How Programmers Interact with Code-Generating Models”, *Proceedings of the ACM on Programming Languages*, t. 7, nr OOPSLA1, 2023, doi: [10.1145/3586030](https://doi.org/10.1145/3586030).
- [4] S. Peng, E. Kalliamvakou, P. Cihon, i M. Demirer, „The Impact of AI on Developer Productivity: Evidence from GitHub Copilot”. 2023. doi: [10.48550/arXiv.2302.06590](https://doi.org/10.48550/arXiv.2302.06590).
- [5] C. E. Jimenez *et al.*, „SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, w *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Dostępne na: <https://openreview.net/forum?id=VTF8yNQm66>
- [6] S. Yao *et al.*, „ReAct: Synergizing Reasoning and Acting in Language Models”, w *International Conference on Learning Representations*, 2023. [Online]. Dostępne na: https://openreview.net/forum?id=WE_vluYUL-X
- [7] J. Yang *et al.*, „SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”, w *Advances in Neural Information Processing Systems*, 2024. doi: [10.52202/079017-1601](https://doi.org/10.52202/079017-1601).
- [8] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, i A. P. Barros, „Workflow Patterns”, *Distributed and Parallel Databases*, t. 14, nr 1, s. 5–51, 2003, doi: [10.1023/A:1022883727209](https://doi.org/10.1023/A:1022883727209).
- [9] T. Berners-Lee, R. Fielding, i L. Masinter, „Uniform Resource Identifier (URI): Generic Syntax”, technical report RFC3986, 2005. doi: [10.17487/RFC3986](https://doi.org/10.17487/RFC3986).
- [10] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, Doctoral dissertation, 2000. [Online]. Dostępne na: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [11] Kancelaria Sejmu, „Kodeks cywilny — tekst jednolity”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://eli.gov.pl/eli/DU/2026/795/ogl>
- [12] Państwowa Inspekcja Pracy, „Jakie są różnice pomiędzy umową o pracę a umowami cywilnoprawnymi?”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://www.pip.gov.pl/dla-pracodawcow/pytania-i-odpowiedzi/jakie-sa-roznice-pomiedzy-umowa-o-prace-a-umowami-cywilnoprawnymi>
- [13] Kancelaria Sejmu, „Ustawa z dnia 26 czerwca 1974 r. — Kodeks pracy”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://eli.gov.pl/eli/DU/1974/141/ogl>
- [14] Kancelaria Sejmu, „Ustawa o zatrudnianiu pracowników tymczasowych — tekst jednolity”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://eli.gov.pl/eli/DU/2023/1110/ogl>
- [15] Urząd Ochrony Danych Osobowych, „Przesłanki przetwarzania danych osobowych przy monitoringu pracowników”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://uodo.gov.pl/pl/674/4212>
- [16] Państwowa Inspekcja Pracy, „Formy zatrudnienia oraz podstawowe prawa i obowiązki”. Dostęp: 24 lipca 2026. [Online]. Dostępne na: <https://www.pip.gov.pl/dla-pracownikow/niezbudnik-pracownika/formy-zatrudnienia-oraz-podstawowe-prawa-i-obowiazki>
- [17] F. Dell'Acqua *et al.*, „Navigating the Jagged Technological Frontier: Field Experimental Evidence of the Effects of Artificial Intelligence on Knowledge Worker Productivity and Quality”, *Organization Science*, t. 37, nr 2, s. 403–423, 2026, doi: [10.1287/orsc.2025.21838](https://doi.org/10.1287/orsc.2025.21838).